

Chapter 55 — System Design Lite / API Design for Coding Interviews

0. What You Will Learn in This Chapter

System Design Lite problems sit between pure coding and full system design. They usually ask you to design a small service, API, or object-oriented system with clear operations.

These problems test whether you can:

- define clean APIs
- maintain internal state
- choose appropriate data structures
- reason about scalability
- handle edge cases
- separate responsibilities into classes
- explain tradeoffs
- design for extensibility
- simulate real-world systems

This chapter is not full distributed system design.

Instead, it focuses on coding-interview-style design problems such as:

- Rate Limiter
- Log Storage System
- Leaderboard
- Autocomplete System
- TinyURL
- Parking Lot
- Elevator
- Vending Machine
- File System
- In-memory database
- Consistent Hashing preview
- API design thinking

By the end of this chapter, you should understand:

- Object-oriented design mindset
- API design requirements
- State management
- Modular class design
- Choosing internal data structures
- How to discuss scalability in coding rounds
- How to implement rate limiters
- How to design log storage
- How to design leaderboard
- How to design autocomplete
- How to design TinyURL
- How to approach parking lot / elevator / vending machine problems
- How to explain tradeoffs clearly

Core problems covered in this chapter:

1. Design Logger Rate Limiter
2. Design Hit Counter

3. Design Leaderboard
 4. Design Log Storage System
 5. Design Search Autocomplete System
 6. Encode and Decode TinyURL
 7. Design File System
 8. Design In-Memory File System
 9. Design Underground System
 10. Design Parking System
 11. Design Parking Lot, OOD preview
 12. Design Vending Machine, OOD preview
 13. Design Elevator System, OOD preview
 14. Design Browser History
 15. Design Bounded Blocking Queue, concurrency preview
 16. Consistent Hashing preview
-

1. What Is System Design Lite?

System Design Lite means designing a small, self-contained system with code. Unlike normal algorithm problems, you are not just writing one function. You are designing:

classes
methods
state
data structures
behavior over time

Example:

```
class Leaderboard {  
public:  
    void addScore(int playerId, int score);  
    int top(int K);  
    void reset(int playerId);  
};
```

You need to decide:

- How to store player scores
 - How to get top K efficiently
 - How to reset players
 - What happens with duplicate scores
 - How operations scale
-

2. System Design Lite vs Full System Design

Scope	one class / small service	distributed architecture
Main focus	APIs + data structures	scalability, reliability, storage
Code required	usually yes	usually no or pseudocode
Examples	TinyURL class, Leaderboard	design Twitter, Uber, YouTube
Constraints	LeetCode-style operations	traffic, databases, caches
Interview goal	implement cleanly	reason at system scale

This chapter focuses on coding-round design.

But you should still mention scalability tradeoffs when useful.

3. Core Design Mindset

For every design question, ask:

17. **What are the APIs?**
18. **What state must persist between calls?**
19. **What are the required time complexities?**
20. **Which operations are frequent?**
21. **Which data structures support those operations?**
22. **What edge cases can break the design?**
23. **How would this scale if data grows?**

Good design starts with requirements.

Do not jump into code too early.

4. API Design Checklist

Before coding, clarify:

4.1 Inputs and Outputs

Example:

```
addScore(playerId, score)
```

Does score replace existing score or add to it?

For LeetCode Leaderboard, it adds.

4.2 Constraints

Ask:

How many calls?

How many users?

Do timestamps increase?

Can IDs repeat?

Can values be negative?

4.3 Edge Cases

Examples:

- empty data
- duplicate keys
- missing user
- reset unknown user
- timestamp out of order
- capacity zero
- invalid path
- duplicate creation

4.4 Performance

Ask:

Should get be $O(1)$?

Should top K be $O(K \log n)$?

Can update be $O(\log n)$?

5. Object-Oriented Design Mindset

Even in coding interviews, design should be modular.

A clean OOD solution has:

- meaningful classes
- clear responsibilities
- private internal state
- public API methods
- helper methods
- minimal duplicated logic

Example:

For parking lot:

ParkingLot
ParkingFloor
ParkingSpot
Vehicle
Ticket
Payment

For coding rounds, you may implement simplified versions, but showing good decomposition helps.

6. Common System Design Lite Patterns

rate limiting	hashmap + timestamp queue

Requirement	Common Pattern
top K scores	heap / ordered map / multiset
log search by time	timestamp normalization + range query
autocomplete	trie + top suggestions
tiny URL	id counter + base62
path system	trie/tree of components
recent events	queue / circular buffer
average travel time	map of check-ins + route stats
browser history	vector + current pointer
parking system	counts / spot allocation
elevator	state machine + priority queues
vending machine	inventory + payment state

7. Pattern Recognition Checklist

Ask:

24. Is this really an API design class problem?
25. What state persists across method calls?
26. Is lookup by ID needed?
27. Is ordering by time needed?
28. Is top K needed?
29. Is prefix search needed?
30. Is path hierarchy needed?
31. Is there a capacity or quota?
32. Is randomness involved?
33. Is this a simulation with entities?
34. Are there multiple object types?
35. Is scalability part of the discussion?

8. Full Problem Walkthrough 1 — Logger Rate Limiter

8.1 Problem Summary

Design logger that receives messages with timestamps.

Print message only if it has not been printed in the last 10 seconds.

API:

```
bool shouldPrintMessage(int timestamp, string message);
```

8.2 Key Idea

Store last printed timestamp for each message.

If:

```
timestamp - lastPrinted >= 10
```

then print and update.

8.3 C++ Code

```
class Logger {
private:
    unordered_map<string, int> lastPrinted;

public:
    Logger() {
    }

    bool shouldPrintMessage(int timestamp, string message) {
        if (!lastPrinted.count(message)) {
            lastPrinted[message] = timestamp;
            return true;
        }

        if (timestamp - lastPrinted[message] >= 10) {
            lastPrinted[message] = timestamp;
            return true;
        }

        return false;
    }
};
```

8.4 Complexity

Time: $O(1)$ average

Space: $O(\text{number of unique messages})$

8.5 Scalability Note

If messages are unbounded, old messages can be cleaned using a queue of timestamp-message pairs.

9. Full Problem Walkthrough 2 — Hit Counter

9.1 Problem Summary

Design hit counter.

API:

```
void hit(int timestamp);  
int getHits(int timestamp);
```

Return hits in past 300 seconds.

9.2 Queue Approach

Store timestamps of hits.

On query, remove timestamps older than 300 seconds.

9.3 C++ Code

```
class HitCounter {  
private:  
    queue<int> q;  
  
public:  
    HitCounter() {  
    }  
  
    void hit(int timestamp) {  
        q.push(timestamp);  
    }  
  
    int getHits(int timestamp) {  
        while (!q.empty() && q.front() <= timestamp - 300) {  
            q.pop();  
        }  
  
        return q.size();  
    }  
};
```

9.4 Complexity

Amortized:

```
hit: O(1)  
getHits: O(1) amortized
```

Space:

O(number of hits in last 300 seconds)

9.5 High-Volume Version

Use 300 buckets:

```
class HitCounter {  
private:  
    vector<int> times;  
    vector<int> counts;
```

```

public:
    HitCounter() {
        times.assign(300, 0);
        counts.assign(300, 0);
    }

    void hit(int timestamp) {
        int index = timestamp % 300;

        if (times[index] != timestamp) {
            times[index] = timestamp;
            counts[index] = 1;
        } else {
            counts[index]++;
        }
    }

    int getHits(int timestamp) {
        int total = 0;

        for (int i = 0; i < 300; i++) {
            if (timestamp - times[i] < 300) {
                total += counts[i];
            }
        }

        return total;
    }
};

```

10. Full Problem Walkthrough 3 — Design Leaderboard

10.1 Problem Summary

Support:

```

void addScore(int playerId, int score);
int top(int K);
void reset(int playerId);

```

10.2 Approach 1 — Hash Map + Min Heap for top(K)

Store:

playerId -> score

For top K, scan all scores and maintain min heap of size K.

Good when updates are frequent and top queries are less frequent.

10.3 C++ Code

```
class Leaderboard {
private:
    unordered_map<int, int> scores;

public:
    Leaderboard() {
    }

    void addScore(int playerId, int score) {
        scores[playerId] += score;
    }

    int top(int K) {
        priority_queue<int, vector<int>, greater<int>> minHeap;

        for (auto& [player, score] : scores) {
            minHeap.push(score);

            if (minHeap.size() > K) {
                minHeap.pop();
            }
        }

        int sum = 0;

        while (!minHeap.empty()) {
            sum += minHeap.top();
            minHeap.pop();
        }

        return sum;
    }

    void reset(int playerId) {
        scores.erase(playerId);
    }
};
```

```
}  
};
```

10.4 Complexity

addScore: $O(1)$
reset: $O(1)$
top: $O(n \log K)$

10.5 Alternative — Ordered Multiset

If top is frequent, maintain a sorted multiset of scores.

```
class Leaderboard {  
private:  
    unordered_map<int, int> scores;  
    multiset<int> ordered;  
  
public:  
    void addScore(int playerId, int score) {  
        if (scores.count(playerId)) {  
            ordered.erase(ordered.find(scores[playerId]));  
        }  
  
        scores[playerId] += score;  
        ordered.insert(scores[playerId]);  
    }  
  
    int top(int K) {  
        int sum = 0;  
        auto it = ordered.rbegin();  
  
        while (K-- && it != ordered.rend()) {  
            sum += *it;  
            it++;  
        }  
  
        return sum;  
    }  
  
    void reset(int playerId) {  
        if (!scores.count(playerId)) {  
            return;  
        }  
  
        ordered.erase(ordered.find(scores[playerId]));  
    }  
};
```

```
scores.erase(playerId);  
}  
};
```

This gives:

```
addScore: O(log n)  
reset: O(log n)  
top: O(K)
```

11. Full Problem Walkthrough 4 — Design Log Storage System

11.1 Problem Summary

Store logs with timestamp string:

```
"2017:01:01:23:59:59"
```

APIs:

```
void put(int id, string timestamp);  
vector<int> retrieve(string start, string end, string granularity);
```

Granularity can be:

```
Year, Month, Day, Hour, Minute, Second
```

Return IDs whose timestamps fall within range at that granularity.

11.2 Key Idea

A timestamp is lexicographically sortable if fixed-width.

Use substring prefix based on granularity.

Map granularity to prefix length:

```
Year -> 4  
Month -> 7  
Day -> 10  
Hour -> 13  
Minute -> 16  
Second -> 19
```

11.3 C++ Code

```
class LogSystem {  
private:  
    vector<pair<string, int>> logs;  
    unordered_map<string, int> length;  
  
public:  
    LogSystem() {
```

```

length["Year"] = 4;
length["Month"] = 7;
length["Day"] = 10;
length["Hour"] = 13;
length["Minute"] = 16;
length["Second"] = 19;
}

void put(int id, string timestamp) {
    logs.push_back({timestamp, id});
}

vector<int> retrieve(string start, string end, string granularity) {
    int len = length[granularity];

    string s = start.substr(0, len);
    string e = end.substr(0, len);

    vector<int> result;

    for (auto& [timestamp, id] : logs) {
        string t = timestamp.substr(0, len);

        if (t >= s && t <= e) {
            result.push_back(id);
        }
    }

    return result;
}
};

```

11.4 Complexity

put: $O(1)$
 retrieve: $O(\text{number of logs})$

11.5 Scalability Note

For many queries, store logs in ordered map keyed by timestamp and use range queries.

12. Full Problem Walkthrough 5 — Design Search Autocomplete System

12.1 Problem Summary

Implement autocomplete.

Input historical sentences with frequencies.

For each typed character:

- If character is #, finish sentence and update frequency.
- Otherwise return top 3 matching sentences by prefix.

Ranking:

36. Higher frequency first
37. Lexicographically smaller sentence if tie

12.2 Simple Approach

Use a Trie.

Each Trie node stores all sentence frequencies below that prefix.

For each input character, move in Trie and compute top 3.

This is simple but can be memory-heavy.

12.3 Trie Node

```
struct TrieNode {
    unordered_map<char, TrieNode*> children;
    unordered_map<string, int> counts;
};
```

At insertion, update counts on every prefix node.

12.4 C++ Code

```
class AutocompleteSystem {
private:
    struct TrieNode {
        unordered_map<char, TrieNode*> children;
        unordered_map<string, int> counts;
    };

    TrieNode* root;
    TrieNode* currentNode;
    string currentInput;

    void insert(string sentence, int count) {
        TrieNode* node = root;

        for (char c : sentence) {
            if (!node->children.count(c)) {
```

```

        node->children[c] = new TrieNode();
    }

    node = node->children[c];
    node->counts[sentence] += count;
}
}

```

```

vector<string> top3(TrieNode* node) {
    vector<pair<string, int>> arr;

    for (auto& [sentence, count] : node->counts) {
        arr.push_back({sentence, count});
    }

    sort(arr.begin(), arr.end(), [](auto& a, auto& b) {
        if (a.second != b.second) {
            return a.second > b.second;
        }

        return a.first < b.first;
    });

    vector<string> result;

    for (int i = 0; i < arr.size() && i < 3; i++) {
        result.push_back(arr[i].first);
    }

    return result;
}

```

public:

```

AutocompleteSystem(vector<string>& sentences, vector<int>& times) {
    root = new TrieNode();

    for (int i = 0; i < sentences.size(); i++) {
        insert(sentences[i], times[i]);
    }

    currentNode = root;
    currentInput = "";
}

```

```

    }

    vector<string> input(char c) {
        if (c == '#') {
            insert(currentInput, 1);
            currentInput = "";
            currentNode = root;
            return {};
        }

        currentInput.push_back(c);

        if (currentNode && currentNode->children.count(c)) {
            currentNode = currentNode->children[c];
            return top3(currentNode);
        }

        currentNode = nullptr;
        return {};
    }
};

```

12.5 Complexity

Insertion:

$O(\text{sentence length})$

Input query:

$O(\text{number of sentences under prefix} * \log \text{count})$

12.6 Optimization

Store top 3 directly at each node.

Then each input is closer to:

$O(1)$ or $O(\text{prefix char traversal})$

But updates become more complex.

13. Full Problem Walkthrough 6 — Encode and Decode TinyURL

13.1 Problem Summary

Design methods:

```

string encode(string longUrl);
string decode(string shortUrl);

```

13.2 Simple Counter Approach

Assign each long URL a unique integer ID.

Convert ID to base62.

Store:

code -> longUrl

longUrl -> code

13.3 Base62 Characters

0-9

a-z

A-Z

Total:

62 characters

13.4 C++ Code

```
class Solution {
private:
    unordered_map<string, string> codeToUrl;
    unordered_map<string, string> urlToCode;
    long long id;
    string chars;

    string toBase62(long long x) {
        if (x == 0) {
            return "0";
        }

        string code;

        while (x > 0) {
            code.push_back(chars[x % 62]);
            x /= 62;
        }

        reverse(code.begin(), code.end());

        return code;
    }

public:
    Solution() {
        id = 1;
```

```

    chars =
"0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";
}

string encode(string longUrl) {
    if (urlToCode.count(longUrl)) {
        return "http://tinyurl.com/" + urlToCode[longUrl];
    }

    string code = toBase62(id++);
    urlToCode[longUrl] = code;
    codeToUrl[code] = longUrl;

    return "http://tinyurl.com/" + code;
}

string decode(string shortUrl) {
    string prefix = "http://tinyurl.com/";
    string code = shortUrl.substr(prefix.size());

    return codeToUrl[code];
}
};

```

13.5 Scalability Discussion

In real systems:

- use distributed ID generator
- store mappings in database
- cache hot URLs
- handle collisions if random codes are used
- validate URLs
- support expiration and analytics
- avoid sequential IDs if privacy matters

14. Full Problem Walkthrough 7 — Design File System

14.1 Problem Summary

Support:

```

bool createPath(string path, int value);
int get(string path);

```

Rules:

- Path must start with /.

- Parent path must exist.
- Cannot create duplicate path.

14.2 Key Idea

Use a trie of path components.

14.3 C++ Code

```
class FileSystem {
private:
    struct Node {
        unordered_map<string, Node*> children;
        int value;

        Node() {
            value = -1;
        }
    };

    Node* root;

    vector<string> split(string path) {
        vector<string> parts;
        string current;

        for (char c : path) {
            if (c == '/') {
                if (!current.empty()) {
                    parts.push_back(current);
                    current.clear();
                }
            } else {
                current.push_back(c);
            }
        }

        if (!current.empty()) {
            parts.push_back(current);
        }

        return parts;
    }

public:
    FileSystem() {
```

```

    root = new Node();
}

bool createPath(string path, int value) {
    vector<string> parts = split(path);

    if (parts.empty()) {
        return false;
    }

    Node* node = root;

    for (int i = 0; i < parts.size(); i++) {
        string part = parts[i];

        if (i == parts.size() - 1) {
            if (node->children.count(part)) {
                return false;
            }

            node->children[part] = new Node();
            node->children[part]->value = value;
            return true;
        }

        if (!node->children.count(part)) {
            return false;
        }

        node = node->children[part];
    }

    return false;
}

int get(string path) {
    vector<string> parts = split(path);

    Node* node = root;

    for (string part : parts) {
        if (!node->children.count(part)) {

```

```

        return -1;
    }

    node = node->children[part];
}

return node->value;
}
};

```

15. Full Problem Walkthrough 8 — Design In-Memory File System

15.1 Problem Summary

Support:

```

vector<string> ls(string path);
void mkdir(string path);
void addContentToFile(string filePath, string content);
string readContentFromFile(string filePath);

```

15.2 Key Idea

Use a tree.

Each node can be:

- directory
- file

Directory stores children.

File stores content.

Use map for children so ls returns sorted names.

15.3 C++ Code

```

class FileSystem {
private:
    struct Node {
        bool isFile;
        string content;
        map<string, Node*> children;

        Node() {
            isFile = false;
            content = "";
        }
    };
};

```

```
Node* root;
```

```
vector<string> split(string path) {  
    vector<string> parts;  
    string current;  
  
    for (char c : path) {  
        if (c == '/') {  
            if (!current.empty()) {  
                parts.push_back(current);  
                current.clear();  
            }  
        } else {  
            current.push_back(c);  
        }  
    }  
  
    if (!current.empty()) {  
        parts.push_back(current);  
    }  
  
    return parts;  
}
```

```
Node* traverseCreate(string path) {  
    vector<string> parts = split(path);  
  
    Node* node = root;  
  
    for (string part : parts) {  
        if (!node->children.count(part)) {  
            node->children[part] = new Node();  
        }  
  
        node = node->children[part];  
    }  
  
    return node;  
}
```

```
Node* traverse(string path) {  
    vector<string> parts = split(path);
```

```

    Node* node = root;

    for (string part : parts) {
        node = node->children[part];
    }

    return node;
}

public:
    FileSystem() {
        root = new Node();
    }

    vector<string> ls(string path) {
        Node* node = traverse(path);

        vector<string> result;

        if (node->isFile) {
            vector<string> parts = split(path);
            result.push_back(parts.back());
            return result;
        }

        for (auto& [name, child] : node->children) {
            result.push_back(name);
        }

        return result;
    }

    void mkdir(string path) {
        traverseCreate(path);
    }

    void addContentToFile(string filePath, string content) {
        Node* node = traverseCreate(filePath);
        node->isFile = true;
        node->content += content;
    }
}

```

```
string readContentFromFile(string filePath) {  
    Node* node = traverse(filePath);  
    return node->content;  
}  
};
```

16. Full Problem Walkthrough 9 — Design Underground System

16.1 Problem Summary

Support:

```
checkIn(id, stationName, t)  
checkOut(id, stationName, t)  
getAverageTime(startStation, endStation)
```

16.2 Key Structures

Need:

```
id -> check-in station and time  
route -> total travel time and trip count
```

Route key can be:

```
start + "#" + end
```

16.3 C++ Code

```
class UndergroundSystem {  
private:  
    unordered_map<int, pair<string, int>> checkIns;  
    unordered_map<string, pair<long long, int>> stats;  
  
    string makeKey(string start, string end) {  
        return start + "#" + end;  
    }  
  
public:  
    UndergroundSystem() {  
  
    }  
  
    void checkIn(int id, string stationName, int t) {  
        checkIns[id] = {stationName, t};  
    }  
};
```

```

void checkOut(int id, string stationName, int t) {
    auto [startStation, startTime] = checkIns[id];
    checkIns.erase(id);

    string route = makeKey(startStation, stationName);

    stats[route].first += t - startTime;
    stats[route].second++;
}

double getAverageTime(string startStation, string endStation) {
    string route = makeKey(startStation, endStation);

    auto [total, count] = stats[route];

    return (double)total / count;
}
};

```

17. Full Problem Walkthrough 10 — Design Parking System

17.1 Problem Summary

Support:

```
bool addCar(int carType);
```

Car types:

```

1 = big
2 = medium
3 = small

```

Return true if there is space.

17.2 Simple Count Design

```

class ParkingSystem {
private:
    vector<int> spaces;

public:
    ParkingSystem(int big, int medium, int small) {
        spaces = {0, big, medium, small};
    }
}

```

```
bool addCar(int carType) {  
    if (spaces[carType] == 0) {  
        return false;  
    }  
  
    spaces[carType]--;  
    return true;  
}  
};
```

18. OOD Preview 1 — Parking Lot

A more realistic parking lot design includes:

18.1 Entities

ParkingLot
ParkingFloor
ParkingSpot
Vehicle
Ticket
Payment
Gate

18.2 Possible Class Design

```
enum class VehicleType {  
    Motorcycle,  
    Car,  
    Truck  
};  
  
enum class SpotType {  
    Small,  
    Medium,  
    Large  
};  
  
class Vehicle {  
public:  
    string licensePlate;  
    VehicleType type;  
};  
  
class ParkingSpot {
```

```
public:
    int id;
    SpotType type;
    bool occupied;
    Vehicle* vehicle;
};

class ParkingFloor {
public:
    int floorNumber;
    vector<ParkingSpot> spots;
};

class ParkingLot {
public:
    vector<ParkingFloor> floors;

    bool parkVehicle(Vehicle vehicle);
    bool removeVehicle(string licensePlate);
};
```

18.3 Interview Discussion Points

Mention:

- spot allocation strategy
 - nearest available spot
 - multiple floors
 - ticket generation
 - payment system
 - vehicle-size compatibility
 - concurrency at gates
-

19. OOD Preview 2 — Vending Machine

19.1 Entities

```
VendingMachine
Item
Inventory
Slot
PaymentProcessor
Transaction
```

19.2 State Machine

Vending machine naturally has states:

Idle
SelectingItem
AcceptingPayment
Dispensing
OutOfStock

19.3 Simplified Class Sketch

```
class Item {
public:
    string name;
    int price;
};

class Slot {
public:
    Item item;
    int quantity;
};

class VendingMachine {
private:
    unordered_map<string, Slot> inventory;
    int balance;

public:
    void addItem(string code, Item item, int quantity);
    void insertCoin(int amount);
    bool selectItem(string code);
    int refund();
};
```

19.4 Discussion Points

- insufficient balance
 - out of stock
 - change return
 - payment types
 - admin restocking
 - transaction cancellation
-

20. OOD Preview 3 — Elevator System

20.1 Entities

ElevatorSystem
Elevator
Request
Floor
Scheduler

20.2 Elevator State

```
enum class Direction {  
    Up,  
    Down,  
    Idle  
};
```

```
class Elevator {  
public:  
    int id;  
    int currentFloor;  
    Direction direction;  
    set<int> targets;  
};
```

20.3 Scheduling Strategies

Possible strategies:

- nearest elevator
- same direction first
- load balancing
- separate up/down request queues
- SCAN algorithm, like disk scheduling

20.4 Interview Discussion Points

Elevator design is often more about tradeoffs than final code.

Mention:

- internal requests vs external floor requests
 - direction-aware scheduling
 - capacity limits
 - emergency state
 - multiple elevators
 - fairness and starvation
-

21. Consistent Hashing Preview

21.1 Problem

Suppose you have many keys and multiple servers.

Naive hashing:

```
server = hash(key) % numberOfServers
```

Problem:

If number of servers changes, most keys remap.

21.2 Consistent Hashing Idea

Place servers and keys on a hash ring.

Each key belongs to next server clockwise.

When a server is added or removed, only nearby keys move.

21.3 Virtual Nodes

To balance load, each physical server appears multiple times on ring.

```
serverA#1
```

```
serverA#2
```

```
serverA#3
```

21.4 When to Mention

Use consistent hashing when discussing:

- distributed cache
- sharding
- TinyURL storage
- load distribution
- scalable key-value stores

22. Rate Limiter Design Patterns

Rate limiters are common System Design Lite questions.

22.1 Fixed Window Counter

Allow at most N requests per fixed time window.

Example:

```
100 requests per minute
```

Use:

```
user -> windowStart, count
```

Simple but boundary problem exists.

22.2 Sliding Window Log

Store timestamps of requests per user.

Remove old timestamps.

Accurate but memory-heavy.

22.3 Sliding Window Counter

Approximate sliding window using current and previous windows.

Less memory, smoother than fixed window.

22.4 Token Bucket

Tokens refill over time.

Each request consumes one token.

Allows bursts up to bucket capacity.

23. Rate Limiter Implementation — Sliding Window Log

23.1 Problem

Allow each user at most limit requests in past window seconds.

23.2 C++ Code

```
class RateLimiter {
private:
    int limit;
    int window;
    unordered_map<string, deque<int>> requests;

public:
    RateLimiter(int limit, int window) {
        this->limit = limit;
        this->window = window;
    }

    bool allowRequest(string userId, int timestamp) {
        deque<int>& q = requests[userId];

        while (!q.empty() && q.front() <= timestamp - window) {
            q.pop_front();
        }

        if (q.size() >= limit) {
            return false;
        }

        q.push_back(timestamp);
        return true;
    }
};
```

23.3 Complexity

Each request timestamp is pushed once and popped once.
Amortized:

$O(1)$

Space:

$O(\text{active users} * \text{limit})$

24. Rate Limiter Implementation — Token Bucket

24.1 Idea

Each user has tokens.

- Bucket capacity = max burst
- Refill rate = tokens per second
- Request allowed if at least 1 token

24.2 C++ Sketch

```
class TokenBucketRateLimiter {
private:
    struct Bucket {
        double tokens;
        int lastTime;
    };

    int capacity;
    double refillRate;
    unordered_map<string, Bucket> buckets;

public:
    TokenBucketRateLimiter(int capacity, double refillRate) {
        this->capacity = capacity;
        this->refillRate = refillRate;
    }

    bool allowRequest(string userId, int timestamp) {
        if (!buckets.count(userId)) {
            buckets[userId] = {(double)capacity, timestamp};
        }

        Bucket& bucket = buckets[userId];

        double elapsed = timestamp - bucket.lastTime;
        bucket.tokens = min((double)capacity, bucket.tokens + elapsed * refillRate);
        bucket.lastTime = timestamp;

        if (bucket.tokens >= 1.0) {
            bucket.tokens -= 1.0;
            return true;
        }
    }
};
```

```
    }  
  
    return false;  
}  
};
```

24.3 Discussion

Token bucket allows bursts.

Sliding log enforces exact request count in a moving window.

25. TinyURL Design Discussion

For coding interviews, simple map + counter is enough.

For system design discussion, mention:

25.1 Code Generation Options

Counter + Base62

Pros:

- no collisions
- simple
- compact

Cons:

- predictable
- needs distributed ID coordination

Random Code

Pros:

- less predictable

Cons:

- collision handling needed

Hash URL

Pros:

- deterministic

Cons:

- collisions possible
- same URL maps same unless salted

25.2 Storage

Need:

shortCode -> longURL

Optionally:

longURL -> shortCode

25.3 Scaling

Discuss:

- database indexing by short code
- caching hot short URLs
- replication
- expiration
- analytics

- abuse prevention
-

26. Autocomplete Design Discussion

26.1 Trie-Based

Pros:

- natural prefix lookup
- fast prefix traversal

Cons:

- memory-heavy
- updates can be expensive if storing top K at each node

26.2 Hash Map Prefix-Based

Precompute:

prefix -> top suggestions

Pros:

- very fast query

Cons:

- huge memory
- expensive updates

26.3 Search Engine Style

For large-scale system:

- query logs
- offline ranking
- prefix index
- caching popular prefixes
- personalized ranking

For coding rounds, Trie is usually expected.

27. Leaderboard Design Discussion

27.1 Small/Medium Scale

Use:

hashmap player -> score
sorted set / heap for ranking

27.2 Frequent Updates

Use balanced tree / multiset.

27.3 Frequent Top K Queries

Maintain ordered scores.

27.4 Large Scale

Mention:

- sharding by player ID
- approximate top K with streaming algorithms

- Redis sorted sets
 - periodic aggregation
-

28. Common API Design Edge Cases

28.1 Duplicate Creation

File path or user creation may fail if already exists.

28.2 Missing Key

Return:

-1, "", false, or exception

depending on problem.

28.3 Timestamp Order

Some problems guarantee increasing timestamps.

If not guaranteed, you may need sorting or ordered maps.

28.4 Capacity Zero

Caches and rate limiters may reject everything.

28.5 Repeated Updates

Should update existing state or create new version?

Clarify.

28.6 Concurrent Calls

Usually ignored in coding rounds, but mention locks for real systems.

29. Common System Design Lite Mistakes

Mistake 1 — Not Clarifying API Semantics

Example:

Does addScore add or replace?

Mistake 2 — Ignoring Operation Complexity

A design may work functionally but fail required constraints.

Mistake 3 — Overengineering

Do not build a distributed architecture for a LeetCode class problem.

Mistake 4 — Underexplaining Tradeoffs

For design questions, explanation matters.

Mistake 5 — Poor State Separation

Mixing all logic in one method makes bugs likely.

Mistake 6 — Missing Edge Cases Across Calls

Design bugs usually appear after sequences:

add, remove, add again, query

30. Interview Explanation Templates

30.1 General Design

I first identify the operations and required complexity. Then I choose internal data structures that make the most frequent or constrained operations efficient. I keep state private and update all structures consistently after each API call.

30.2 Rate Limiter

For exact sliding-window rate limiting, I keep a queue of request timestamps per user and evict timestamps outside the window before deciding whether to allow a new request.

30.3 TinyURL

I assign each long URL a unique ID and encode it using base62 to generate a short code. I store mappings in hash maps for $O(1)$ encode/decode.

30.4 Leaderboard

I store each player's score in a hash map. If top queries are infrequent, I compute top K with a heap. If top queries are frequent, I maintain an ordered multiset of scores.

30.5 Autocomplete

I use a Trie because autocomplete is prefix-based. Each node represents a prefix and stores candidate sentences or top suggestions for that prefix.

31. Complexity Patterns

Logger Rate Limiter	hashmap	$O(1)$
Hit Counter	queue / buckets	$O(1)$ amortized
Leaderboard heap version	hashmap + heap	top $O(n \log K)$
Leaderboard multiset version	hashmap + multiset	add/reset $O(\log n)$, top $O(K)$
Log Storage	vector scan	retrieve $O(n)$
Autocomplete	trie	input depends on prefix candidates
TinyURL	hash maps + counter	$O(1)$
File System	trie/tree	$O(\text{path length})$

Design	Common Structures	Complexity
Underground System	hash maps	$O(1)$
Sliding Rate Limiter	hashmap + deque	$O(1)$ amortized
Token Bucket	hashmap	$O(1)$

32. Expanded Practice Set

32.1 Must-Do Medium

38. Logger Rate Limiter
39. Hit Counter
40. Design Leaderboard
41. Log Storage System
42. Encode and Decode TinyURL
43. Design File System
44. Design Underground System
45. Design Browser History

32.2 Strong Interview Practice

46. Search Autocomplete System
47. In-Memory File System
48. Sliding Window Rate Limiter
49. Token Bucket Rate Limiter
50. Parking Lot OOD
51. Vending Machine OOD
52. Elevator System OOD

32.3 Hard / Stretch

53. Consistent Hashing implementation
 54. Distributed rate limiter discussion
 55. TinyURL scalable design
 56. Leaderboard at scale
 57. Autocomplete at scale
 58. Design Mini In-Memory Database
 59. Design Spreadsheet / Excel Formula
-

33. Suggested Study Order

Step 1 — Simple API State

Solve:

60. Logger Rate Limiter
61. Hit Counter
62. Browser History

Goal:

Understand persistent state across calls.

Step 2 — Hash Map Designs

Solve:

- 63. Underground System
- 64. TinyURL
- 65. Leaderboard

Goal:

Use maps to track entities and aggregates.

Step 3 — Tree / Path Designs

Solve:

- 66. Design File System
- 67. In-Memory File System

Goal:

Model hierarchical paths.

Step 4 — Prefix Search Design

Solve:

- 68. Autocomplete System

Goal:

Use Trie for prefix APIs.

Step 5 — Rate Limiter Variants

Implement:

- 69. Fixed window
- 70. Sliding log
- 71. Token bucket

Goal:

Explain tradeoffs.

Step 6 — OOD Simulations

Sketch:

- 72. Parking Lot
- 73. Vending Machine
- 74. Elevator

Goal:

Practice class decomposition and tradeoffs.

34. Pattern Decision Table

print message every N seconds	hashmap message -> last time
count events in recent window	queue / circular buckets
top K scores	heap or ordered set
range logs by timestamp	sorted timestamps / prefix comparison
autocomplete	trie
shorten URL	counter/random code + maps
file paths	trie/tree

Requirement	Design Pattern
travel average	check-in map + route aggregate
simple parking	capacity counters
real parking lot	OOD entities
exact rate limiting	sliding window log
burst-friendly rate limiting	token bucket
distribute keys across servers	consistent hashing

35. Mini Problem 1 — Fixed Window Rate Limiter

Allow at most N requests per user per fixed window.

```
class FixedWindowRateLimiter {
private:
    int limit;
    int windowSize;

    struct Counter {
        int windowStart;
        int count;
    };

    unordered_map<string, Counter> mp;

public:
    FixedWindowRateLimiter(int limit, int windowSize) {
        this->limit = limit;
        this->windowSize = windowSize;
    }

    bool allow(string userId, int timestamp) {
        int windowStart = timestamp / windowSize * windowSize;

        if (!mp.count(userId) || mp[userId].windowStart != windowStart) {
            mp[userId] = {windowStart, 1};
            return true;
        }
    }
}
```

```
    if (mp[userId].count >= limit) {  
        return false;  
    }  
  
    mp[userId].count++;  
    return true;  
}  
};
```

Weakness:

Boundary bursts can occur near window edges.

36. Mini Problem 2 — Simple In-Memory Key-Value Store

Support:

```
set(key, value)  
get(key)  
delete(key)  
class KeyValueStore {  
private:  
    unordered_map<string, string> mp;  
  
public:  
    void set(string key, string value) {  
        mp[key] = value;  
    }  
  
    string get(string key) {  
        if (!mp.count(key)) {  
            return "";  
        }  
  
        return mp[key];  
    }  
  
    bool remove(string key) {  
        if (!mp.count(key)) {  
            return false;  
        }  
    }
```

```
        mp.erase(key);
        return true;
    }
};
```

Extension ideas:

- TTL expiration
- transactions
- snapshots
- prefix search
- persistence

37. Mini Problem 3 — Key-Value Store With TTL

Each key expires after a timestamp.

```
class TTLStore {
private:
    struct Entry {
        string value;
        int expireAt;
    };

    unordered_map<string, Entry> mp;

public:
    void set(string key, string value, int ttl, int currentTime) {
        mp[key] = {value, currentTime + ttl};
    }

    string get(string key, int currentTime) {
        if (!mp.count(key)) {
            return "";
        }

        if (mp[key].expireAt <= currentTime) {
            mp.erase(key);
            return "";
        }

        return mp[key].value;
    }
};
```

```
}  
};
```

Scalability improvement:

Use min-heap by expiration time for cleanup.

38. Mini Problem 4 — Top K Frequent Words Service

Maintain word counts and return top K.

Simple query-time heap:

```
class WordCounter {  
private:  
    unordered_map<string, int> count;  
  
public:  
    void addWord(string word) {  
        count[word]++;  
    }  
  
    vector<string> topK(int k) {  
        vector<pair<string, int>> arr;  
  
        for (auto& [word, freq] : count) {  
            arr.push_back({word, freq});  
        }  
  
        sort(arr.begin(), arr.end(), [](auto& a, auto& b) {  
            if (a.second != b.second) {  
                return a.second > b.second;  
            }  
  
            return a.first < b.first;  
        });  
  
        vector<string> result;  
  
        for (int i = 0; i < arr.size() && i < k; i++) {  
            result.push_back(arr[i].first);  
        }  
    }  
};
```

```
        return result;
    }
};
```

Tradeoff:

Fast updates, slower topK.

39. Debugging System Design Lite

If your design fails, check:

75. Did you implement the exact API semantics?
76. Does state persist correctly across calls?
77. Are duplicate IDs handled?
78. Are missing keys handled?
79. Are timestamps inclusive or exclusive?
80. Is cleanup happening before or after query?
81. Are data structures synchronized?
82. Does your complexity match the requirement?
83. Did you consider capacity zero?
84. Did you test long operation sequences?

40. How to Explain Tradeoffs

A strong answer includes tradeoffs.

Example for Leaderboard:

If top(K) is rare, I prefer hashmap + heap because updates are $O(1)$. If top(K) is frequent, I maintain a sorted multiset, making updates $O(\log n)$ but top(K) $O(K)$.

Example for Rate Limiter:

Sliding window log is exact but stores request timestamps. Token bucket is more memory efficient and allows controlled bursts, but it models rate rather than exact request count over a window.

Example for TinyURL:

Counter-based codes avoid collisions but are predictable. Random codes are harder to guess but require collision detection.

41. What to Master Before Chapter 56

Before moving to Concurrency Basics, make sure you can:

- Design clean APIs
- Maintain class state across calls
- Choose internal data structures from operation requirements
- Implement Logger Rate Limiter

- Implement Hit Counter
 - Implement Leaderboard
 - Implement Log Storage System
 - Implement TinyURL
 - Implement File System
 - Implement Underground System
 - Explain Rate Limiter variants
 - Sketch Parking Lot OOD
 - Sketch Vending Machine OOD
 - Sketch Elevator OOD
 - Explain scalability tradeoffs without overengineering
-

42. Final Cheat Sheet

Design Process

Clarify APIs
Clarify constraints
Identify persistent state
Choose data structures
Define update rules
Handle edge cases
Discuss tradeoffs

Rate Limiter

Fixed window: simple, boundary burst
Sliding log: exact, more memory
Token bucket: burst-friendly, rate-based

TinyURL

id -> base62 code
code -> longUrl
longUrl -> code optional

Leaderboard

hashmap player -> score
heap for query-time top K
or multiset for frequent top K

Log Storage

fixed-width timestamp strings
compare prefixes by granularity

Autocomplete

Trie for prefix search
ranking by frequency, then lexicographic order

File System

tree/trie of path components
directory nodes contain children
file nodes contain content

Underground System

id -> check-in info
route -> total time, count

OOD

Identify entities
Define responsibilities
Keep state private
Expose clean methods
Discuss extensions

43. Core Takeaway

System Design Lite problems are about turning requirements into clean stateful APIs.

The key is not just knowing data structures.

The key is knowing how to compose them.

Always ask:

What must be fast?
What state do I need?
How do updates affect that state?
What edge cases happen across multiple calls?

For coding interviews, implement the simplest design that satisfies the constraints.

For design discussions, add tradeoffs and scalability notes.

That balance is what interviewers usually want.
